

پروژه یک

اعضای گروه:

امیرحسین صباحی - 810101556

مجید سپاهی - 810100163

آشنایی با سیستم عامل و xv6

1. سه وظیفه اصلی سیستم عامل را نام ببرید.

یک- مدیریت دسترسی به سخت افزار: سیستم عامل وظیفه دارد که سخت افزار کامپیوتر را کنترل و مدیریت کند و جزئیات فنی و پیچیده آن را از دید برنامه ها پنهان سازد.

دو- مدیریت منابع: شامل مدیریت منابع سخت افزاری مانند CPU ، Memory ، دستگاه های ورودی/خروجی و سایر منابع سیستم است تا به طور کارا بین برنامه ها تقسیم شوند.

سه- مدیریت وظایف و فرآیندها: شامل ایجاد، زمان بندی، اجرا، توقف و پایان فرآیندها است و همچنین ارتباط و هماهنگی بین آنها را فراهم می کند.

2. آیا وجود سیستم عامل در تمام دستگاه ها الزامی است؟ چرا؟ در چه شرایطی استفاده از سیستم عامل لازم است؟

خیر- دستگاه های ساده با یک وظیفه خاص مانند ماشین حساب ، بدون سیستم عامل هم کار می کنند. چون کار آنها محدود و از پیش تعیین شده است. دستگاه هایی که چند وظیفه دارند یا باید بین کاربران و برنامه های مختلف هماهنگی برقرار کنند، به سیستم عامل نیاز دارند.

3. معماری سیستم عامل 6xv چیست؟ چه دلایلی در دفاع از نظر خود دارید؟

سیستم عامل 6xv یک نسخه آموزشی و ساده شده از Unix نسخه ششم (6Unix V) است و مبتنی بر پردازنده های 86x نوشته شده است. این سیستم عامل معماری تک هسته ای دارد. سیستم عامل Unix از سه بخش اصلی shell ، kernel و user applications تشکیل شده است. سیستم عامل 6xv نیز همین ساختار را دارد و به طور کلی در اجرای پردازش ها از روش Unix تبعیت می کند.

4. سیستم عامل 6xv یک سیستم تک وظیفه ای است یا چند وظیفه ای؟

چندوظیفه ای- در 6xv، مکانیزم زمان بندی (Scheduler) وجود دارد که تصمیم می گیرد در هر لحظه کدام فرآیند روی CPU اجرا شود. بنابراین 6xv برخلاف سیستم های تک وظیفه ای، از چندوظیفگی پشتیبانی می کند.

5. همانطور که میدانید به طور کلی چندوظیفگی تعمیمی است از حالت چندبرنامگی. چه تفاوتی میان یک برنامه و یک پردازنده وجود دارد؟

برنامه مجموعه ای از دستورالعمل ها یا کدی است که روی حافظه ثانویه (مثل دیسک) ذخیره شده و هنوز اجرا نشده است. در واقع برنامه فقط یک فایل اجرایی یا مجموعه ای از دستورات است.

اما پردازنده نمونه در حال اجرای یک برنامه در حافظه است. وقتی برنامه ای اجرا می شود، سیستم عامل آن را به صورت یک پردازنده با فضای آدرس مخصوص، شمارنده برنامه و منابع خاص مانند حافظه و فایل ها مدیریت می کند.

6. ساختار یک پردازنده در سیستم عامل 6xv از چه بخشهایی تشکیل شده است؟ این سیستم عامل به طور کلی چگونه پردازنده را به پردازنده های مختلف اختصاص می دهد؟

هر پردازنده شامل اطلاعاتی مثل شناسه پردازنده، وضعیت، شمارنده برنامه، فضای آدرس (کد، داده و پشته) و جدول فایل ها است. 6xv با استفاده از زمان بند خود، پردازنده را به صورت نوبتی ((round-robin بین پردازنده های آماده به اجرا تقسیم می کند تا همه آن ها فرصت اجرا داشته باشند.

7. مفهوم file descriptor در سیستم عامل های مبتنی بر UNIX چیست؟ عملکرد pipe در سیستم عامل 6xv چگونه است و به طور معمول برای چه هدفی استفاده می شود؟

file descriptor عددی است که سیستم عامل برای هر فایل باز به یک پردازنده اختصاص می دهد. این عدد در واقع اشاره گر به جدول فایل های باز در هسته است. هر پردازنده معمولاً سه توصیفگر فایل پیش فرض دارد: 0 برای ورودی استاندارد (stdin)، 1 برای خروجی استاندارد (stdout) و 2 برای خطای استاندارد.

(stderr) وقتی پردازش فایلی را باز می‌کند، سیستم عامل یک عدد جدید به آن اختصاص می‌دهد تا از طریق آن بتواند روی فایل عملیات خواندن یا نوشتن انجام دهد.

Pipe مکانیزمی برای برقراری ارتباط بین دو پردازش است. این مکانیزم یک بافر در حافظه ایجاد می‌کند که یک پردازش می‌تواند در آن بنویسد و پردازش دیگر از آن بخواند. به صورت معمول pipe برای انتقال داده ها بین پردازش ها استفاده می شود مثلا وقتی خروجی یک برنامه باید به عنوان ورودی برنامه دیگر نوشته شود.

8. فراخوانی های سیستمی exec و fork در سیستم عامل xv6 چه عملیاتی را انجام میدهند؟ از نظر طراحی، ادغام نکردن این دو چه مزیتی دارد؟

فراخوانی fork باعث می‌شود یک پردازش جدید به وجود آید که کپی دقیقی از پردازش والد است. پردازش جدید همان کد، داده و وضعیت را دارد، اما شناسه پردازش (PID) متفاوتی دارد.

فراخوانی exec فایل اجرایی جدیدی را در فضای همان پردازش بارگذاری می‌کند و محتوای برنامه قبلی را جایگزین می‌سازد، به این ترتیب پردازش همان است ولی برنامه در حال اجرا تغییر می‌کند.

مزیت جدا بودن این دو فراخوانی در طراحی این است که سیستم عامل انعطاف پذیرتر می‌شود؛ یعنی ابتدا می‌توان پردازش جدید را با fork ایجاد کرد، سپس در صورت نیاز برنامه‌ای متفاوت را با exec در آن اجرا نمود. این جداسازی امکان اجرای عملیات میانی (مثل تغییر ورودی و خروجی یا تنظیم محیط پردازش) پیش از اجرای برنامه جدید را فراهم می‌کند.

اجرا و برپایی سیستم عامل xv6 روی شبیه ساز Qemu:

9. دستور make-n را اجرا نمایید. کدام دستور، فایل نهایی هسته را می‌سازد؟

فایل نهایی هسته (kernel) توسط دستور ld ساخته می‌شود که وظیفه‌ی لینک کردن فایل‌های کامپایل شده را بر عهده دارد.

```
ld -m elf_i386 -T kernel.ld -o kernel entry.o bio.o console.o exec.o file.o fs.o ide.o ioapic.o kalloc.o kbd.o lapic.o log.o main.o mp.o picirq.o pipe.o proc.o sleeplock.o spinlock.o string.o swtch.o syscall.o sysfile.o sysproc.o trapasm.o trap.o uart.o vectors.o vm.o -b
```

این دستور هسته‌ی سیستم عامل (kernel) را می‌سازد زیرا از ابزار ld برای لینک کردن تمام فایل‌های مربوط به اجزای هسته (که قبلا از طریق کامپایلر به صورت جداگانه کامپایل شده اند) استفاده می‌کند.

اضافه کردن یک متن به boot message :

```

int
✓ main(void)
{
    int pid, wpid;
    ✓ if(open("console", O_RDWR) < 0){
        mknod("console", 1, 1);
        open("console", O_RDWR);
    }
    dup(0); // stdout
    dup(0); // stderr

    ✓ for(;;){
        printf(1, "init: starting sh\n");
        printf(1, "  Group Members:\n");
        printf(1, "    - Majid Sepahi\n");
        printf(1, "    - AmirHosein sabahi\n");
        pid = fork();
        ✓ if(pid < 0){
            printf(1, "init: fork failed\n");
            exit();
        }
        ✓ if(pid == 0){
            exec("sh", argv);
        }
    }
}

```

افزودن این ویژگی در فایل `init.c` صورت می گیرد و نتیجه ان به صورت زیر است:

```

xv6...
cpu0: starting 0
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap start 58
init: starting sh
  Group Members:
    - Majid Sepahi
    - AmirHosein sabahi
$

```

اضافه کردن قابلیت های جدید به کنسول:

کنسول `6xv` همزمان چند قابلیت تعاملی دریافت کرده است: جابه جایی دقیق کرسر، ویرایش در وسط خط، انتخاب/کپی/پیست، و «بازگردانی آخرین درج». داده خط با سه اندیس مدیریت می شود: **w** ابتدای خط جاری، **e** یکپس از-انتهای متن، و **c** جای منطقی کرسر. همه خروجی ها از مسیر یکتا می گذرند تا بازکشیدن خط همواره سازگار باشد و کاراکتر اضافی روی نمایش ظاهر نشود.

گروه اول توابع، حرکت کرسر و بازترسیم را پوشش می دهد: `ansi_left/right` برای حرکت فیزیکی با دنباله های

`ANSI` و `move_cursor_to` برای رساندن کرسر به مقصد در بازه `[w..e]`. هسته نمایش `redraw_line_at` است:

از موقعیت فیزیکی فعلی تا ابتدای خط می‌آید، کل متن را (با هایلایت انتخاب فعال) می‌نویسد، سپس کرسر را به مقصد برمی‌گرداند. نسخه سادتر `redraw_line` همین کار را با فرض تساوی مبدأ/مقصد انجام می‌دهد.

برای ویرایش، سه پرایمیتو افزوده شده است. `insert_text` ابتدا بخش راست را به‌درستی شیفت می‌دهد، کاراکترهای

جدید را می‌نویسد و جای هر درج را در پشته `ins_pos` ثبت می‌کند تا `Ctrl+Z` فقط «آخرین درج» را برگرداند.

`delete_at_keep_cursor` یک کاراکتر را حذف می‌کند اما جای ظاهری کرسر را ثابت نگه می‌دارد. `delete_range`

برای حذف انتخاب به کار می‌رود و پس از شیفت چپ، اندیس‌ها و پشته درج‌ها را تصحیح می‌کند.

منطق انتخاب/کپی/پیست با حالت‌های `sel_pending` و `sel_active` پیاده شده است. با `Ctrl+S` بار اول نقطه

لنجر تعیین می‌شود و بار دوم بازه `[lo,hi]` فعال و با معکوس‌نما دیده می‌شود. `Ctrl+C` متن انتخابی را به کلیپ‌بورد داخلی می‌برد بدون لغو انتخاب. `Ctrl+V` اگر انتخابی باشد ابتدا آن را حذف و سپس محتوای کلیپ‌بورد

را در محل کرسر درج می‌کند؛ هر تایپ یا `Backspace` در حالت انتخاب نیز همین الگوی «حذف سپس درج» را دارد.

تابع `consoleintr` مرکز پردازش کلیدهاست. ابتدا یک ماشین حالت کوچک برای `ESC` و کلیدهای جهت مدیریت می‌شود،

سپس—با توجه به انتخاب فعال—فقط کلیدهای مجاز پذیرفته می‌شوند و بقیه موجب لغو انتخاب خواهند شد. شاخه‌های

کنترلی شامل `Ctrl+A/D` برای حرکت کرسر، `Ctrl+Z` برای بازگردانی آخرین درج، `Ctrl+U` برای پاک‌سازی خط،

و اینتر برای نهایی‌سازی خط و بیدار کردن خواننده کنسول است. که در تابع `consoleintr` پیاده‌سازی به صورت زیر است:

```

consoleintr(int (*getc)(void)){
    int ch, doprocdump = 0;

    acquire(&cons.lock);
    while((ch = getc()) >= 0){

        // ESC parsing for arrows
        if(input.esc_state == 1){ if(ch == '['){ input.esc_state = 2; continue; } input.esc_state = 0; continue; }
        else if(input.esc_state == 2){
            if(input.sel_active){ selection_clear(); continue; }
            if(ch == 'D'){ if(input.c > input.w){ input.c--; ansi_left(1); } }
            else if(ch == 'C'){ if(input.c < input.e){ input.c++; ansi_right(1); } }
            input.esc_state = 0; continue;
        }
        if(ch == 27){ input.esc_state = 1; continue; }

        if(input.sel_active){
            int keep = 0;
            if(ch == C('C') || ch == C('V') || ch == C('S') ||
               ch == 0x08 || ch == 0x7f || is_printable(ch) || ch == '\r' || ch == '\n') keep = 1;
            if(!keep){ selection_clear(); continue; }
        }

        switch(ch){
        case C('P'):
            doprocdump = 1; break;

        case C('S'):
            if(!input.sel_pending && !input.sel_active){
                input.sel_anchor = input.c; input.sel_pending = 1;
            }else if(input.sel_pending){
                if(input.c != input.sel_anchor){
                    input.sel_lo = input.c < input.sel_anchor ? input.c : input.sel_anchor;
                    input.sel_hi = input.c > input.sel_anchor ? input.c : input.sel_anchor;
                    input.sel_active = 1; input.sel_pending = 0; redraw_line();
                }else input.sel_pending = 0;
            }else selection_clear();
            break;

        case C('U'):
            while(input.e != input.w && input.buf[(input.e-1) % INPUT_BUF] != '\n')
                delete_at_keep_cursor(input.e - 1);
            move_cursor_to(input.w);
            input.ins_top = 0;
            selection_clear();
            break;

        case C('D'):
            if(input.e == input.w){ input.w = input.e; wakeup(&input.r); }
            else move_cursor_to(next_word_start());
            break;

        case C('A'):
            move_cursor_to(prev_word_start());
            break;

```

```

case C('C'):
    if(input.sel_active){
        int n = input.sel_hi - input.sel_lo; if(n > INPUT_BUF-1) n = INPUT_BUF-1;
        for(int i=0;i<n;i++) input.clip[i] = input.buf[(input.sel_lo + i) % INPUT_BUF];
        input.clip_len = n; input.clip[n] = 0;
    }
    break;

case C('V'):
    if(input.clip_len > 0){
        if(input.sel_active){ uint lo=input.sel_lo, hi=input.sel_hi; input.sel_active=0; input.sel_pending=0; delete_range(lo, hi); }
        insert_text(input.clip, input.clip_len);
    }
    break;

case C('H'):
case 0x7f:
    if(input.sel_active){
        uint lo=input.sel_lo, hi=input.sel_hi;
        input.sel_active=0; input.sel_pending=0;
        delete_range(lo, hi);
    }else if(input.c > input.w){
        delete_at_keep_cursor(input.c - 1);
    }
    break;

case C('Z'):
    if(input.sel_active){ selection_clear(); break; }
    while(input.ins_top > 0){
        int pos = input.ins_pos[--input.ins_top];
        if(pos < (int)input.e){ delete_at_keep_cursor((uint)pos); break; }
    }
    break;

default:
    if(ch == '\r') ch = '\n';
    if(ch != 0 && input.e - input.r < INPUT_BUF){
        if(input.sel_active){ uint lo=input.sel_lo, hi=input.sel_hi; input.sel_active=0; input.sel_pending=0; delete_range(lo, hi); }
        if(ch == '\n'){
            input.buf[input.e % INPUT_BUF] = '\n';
            if(input.ins_top < INPUT_BUF) input.ins_pos[input.ins_top++] = input.e;
            input.e++; input.c = input.e;
            consputc('\n');
            input.w = input.e;
            input.ins_top = 0;
            wakeup(&input.r);
        }else{
            char cc = (char)ch;
            insert_text(&cc, 1);
        }
    }
    break;
}
}
release(&cons.lock);
if(doprocdump) procdump();
}

```

برنامه سطح کاربر:

برنامه در فایل find_sum.c نوشته شده و به لیست برنامه های سطح کاربر در makefile اضافه شده است.

نتیجه تست برنامه:

```

qemu-system-i386 -serial mon:stdio -drive file=fs.img,index=1,media=disk,format=raw -driv
sk,format=raw -smp 2 -m 512
xv6...
cpu0: starting 0
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap start 58
init: starting sh
  Group Members:
    - Majid Sepahi
    - AmirHosein sabahi
$ find_sum op1e2rati34ng5 6Sy7ste8m!
$ cat result.txt
63
$

```

01. در Makefile متغیرهایی به نامهای UPROGS و ULIB تعریف شده است.
کاربرد آنها چیست؟

متغیر ULIB شامل فایل‌های کتابخانه‌ای است که در همه‌ی برنامه‌های کاربر استفاده می‌شوند، مانند توابعی برای فراخوانی‌های سیستمی مثل `read`، `write`، `fork` و `exec`. این فایل‌ها معمولاً به صورت کد منبع مشترک در اختیار همه‌ی برنامه‌های کاربر قرار می‌گیرند.

```
ULIB = ulib.o usys.o printf.o umalloc.o
```

متغیر UPROGS لیستی از برنامه‌های اجرایی سطح کاربر است که باید در هنگام ساخت سیستم عامل کامپایل و به سیستم اضافه شوند. برای مثال، برنامه‌هایی مانند `cat`، `ls`، `sh` و `echo` در این متغیر تعریف می‌شوند تا در هنگام ساخت، به صورت فایل اجرایی درون فایل سیستم `6xv` قرار گیرند.

```

UPROGS=\
_cat\
_echo\
_forktest\
_grep\
_init\
_kill\
_ln\
_ls\
_mkdir\
_rm\
_sh\
_stressfs\
_usertests\
_wc\
_zombie\

```

11. اگر به فایل‌های موجود در `6xv` دقت کنید، می‌بینید که فایل‌ی مربوط به دستور `cd`، برخلاف دستوراتی مانند `ls` و `cat` وجود ندارد و این دستور در سطح کاربر اجرا نمی‌شود. توضیح دهید که این دستور `cd` در کجا اجرا می‌شود. به نظر شما دلیل این تفاوت میان دستور `cd` و دستورات دیگر مثل `ls` و `cat` چیست؟

دستور `cd` در سیستم عامل `6xv` مانند بیشتر سیستم های مبتنی بر یونیکس در خود پوسته (`Shell`) اجرا می شود، نه به عنوان یک برنامه ی کاربر جداگانه. دلیل این موضوع این است که دستور `cd` باید دایرکتوری جاری همان پوسته را تغییر دهد. در مقابل، دستوراتی مانند `ls` یا `cat` نیازی به تغییر وضعیت داخلی پوسته ندارند و فقط اجرا می شوند و نتیجه را نمایش می دهند. بنابراین می توانند به صورت برنامه های جداگانه در سطح کاربر پیاده سازی شوند.

قابلیت تکمیل خودکار:

اجرای بوت لودر:

12- در `6xv` در سکتور نخست دیسک قابل بوت، محتوای چه فایلی قرار دارد؟ (راهنمایی: خروجی دستور `make -n` را بررسی نمایید.)

فایل `bootblock` در فرآیند ساخت، کدهای `bootasm.S` و `bootmain.c` اول لینک می شوند با آدرس مبنای (با آدرس مبنای `000x7c`)، خروجی `ELF` موقت ساخته می شود، سپس با `objcopy` به یک باینری خام `۵۱۲` بایتی به نام `bootblock` تبدیل می شود و در سکتور صفر ایمیج دیسک `xv6.img` نوشته می شود. این همان رکورد بوت است که BIOS آن را می خواند و اجرا می کند.

13- برنامه های کامپایل شده در قالب فایل های دودویی نگهداری می شوند. فایل مربوط به بوت نیز دودویی است. نوع این فایل دودویی چیست؟ چرا از این نوع فایل استفاده شده است؟ تفاوت این نوع فایل دودویی با دیگر فایل های دودویی `6xv` چیست؟ این فایل را به زبان قابل فهم انسان (اسمبلی) تبدیل نمایید (راهنما: از ابزار `objdump` استفاده کنید).

نوع فایل بوت `bootblock` باینری خام (`Flat Binary`) بدون هدر می باشد. اندازه اش دقیقاً `۵۱۲` بایت و در پایان دارای امضای بوت `0x55 0xAA` است.

چرایی استفاده: BIOS فقط سکتور اول را بی هیچ درک از فرمت می خواند و مستقیماً از آدرس `00x7C0` اجرا می کند؛ پس کد باید «خام» و در `۵۱۲` بایت جا شود.

تفاوت با بقیه دودویی ها: کرنل (`kernel`) و برنامه های کاربری (`_cat_`, `sh`, ...) همگی `ELF 32-bit` هستند (دارای هدر/سگمنت/سمبل)، چون پس از بوت، لودر و سیستم می توانند `ELF` را لود کنند؛ اما بوت سکتور فقط باید خام باشد.

14 - علت استفاده از دستور objcopy در حین اجرای عملیات make چیست؟
دستور objcopy در جریان عملیات make برای تبدیل فرمت فایل‌های شیء (o.) با اجزای و حذف اطلاعات اضافه (مثل هدرها) استفاده می‌شود. این کار برای بهینه سازی و کاهش حجم فایل و ایجاد نسخه‌های باینری خام مناسب برای شبیه‌سازها /بوت‌لودرها انجام می‌گیرد. پس objcopy فایل‌ها را برای سازگاری با محیط‌های خاص آماده می‌کند.

با اجرای دستور make -n موارد زیر را می‌توانیم ببینیم:
در تولید bootblock از bootblock.o فقط بخش کُد اجرایی برداشته می‌شود و بقیه حذف می‌گردد:

```
objcopy -S -O binary -j .text bootblock.o bootblock
```

در تولید entryother از bootblockother.o همان الگو تکرار می‌شود؛ تنها سکشن .text به خام تبدیل می‌شود:

```
objcopy -S -O binary -j .text bootblockother.o entryother
```

در تولید initcode از initcode.out سوئیچ -j استفاده نمی‌شود؛ بنابراین کل محتوا به باینری خام برمی‌گردد:

```
objcopy -S -O binary initcode.out initcode
```

15- در فایل‌های موجود در 6xv، مشاهده می‌شود که بوت سیستم توسط فایل‌های bootasm.S و bootmain.c صورت می‌گیرد. چرا تنها از C استفاده نشده است؟

bootasm.S در real mode 16-bit اجرا می‌شود، پشته/سگمنت‌ها را تنظیم می‌کند، وقفه‌ها را می‌بندد، و بیت PE را در 0CR می‌گذارد تا CPU به Protected Mode 32-bit سوئیچ کند. این مقدمات با C قابل تنظیم نیست زیرا هنوز محیط ۳۲بیت/استک پایدار نداریم.

پس از آماده‌سازی اولیه، کنترل به bootmain.c (کُد ۳۲بیتی) می‌رسد تا هدر ELF کرنل را بخواند و بخش‌های آن را از دیسک به حافظه لود کند و در نهایت به نقطه ورود کرنل برسد.

16. ثبات عام منظوره، یک ثبات قطعه، یک ثبات وضعیت و یک ثبات کنترلی در معماری 86x را نام برده و وظیفه هر یک را به طور مختصر توضیح دهید.

eax ثبات عام منظوره :
برای انجام محاسبات عمومی، نگهداری مقادیر موقت و انتقال داده‌ها در عملیات حسابی و منطقی استفاده می‌شود.

cs ثبات قطعه :
برای نگهداری آدرس پایه بخش کد برنامه به کار می‌رود و مشخص می‌کند دستورات پردازنده از کدام بخش حافظه خوانده شوند.

eflags : ثبات وضعیت
شامل پرچم‌هایی است که وضعیت فعلی پردازنده را نشان می‌دهند، مثل پرچم صفر، پرچم حمل و پرچم علامت. این پرچم‌ها نتیجه عملیات محاسباتی و منطقی را تعیین می‌کنند.

cr3 ثبات کنترلی :
در حالت صفحه‌بندی برای نگهداری آدرس جدول صفحه استفاده می‌شود و نقش مهمی در مدیریت حافظه مجازی دارد.

17-پردازنده های 86x دارای مدهای مختلفی هستند. هنگام بوت، این پردازنده ها در مد حقیقی قرار داده میشوند؛ مدی که سیستم عامل (MS DOS) در آن اجرا می شد. چرا؟ یک نقص اصلی این مد را بیان نمایید. آیا در پردازنده‌های دیگر مانند ARM یا RISC-V نیز مدها به همین شکل هستند یا خیر؟ توضیح دهید.

در هنگام بوت، پردازنده‌های 86x در مد حقیقی قرار می‌گیرند تا بتوانند با بایوس و سخت‌افزارهای اولیه سیستم که بر اساس معماری قدیمی طراحی شده‌اند ارتباط برقرار کنند. این مد از روزهای اولیه رایانه‌های شخصی باقی مانده تا سازگاری با نرم‌افزارهای قدیمی مانند سیستم عامل MS-DOS حفظ شود. سیستم عامل MS-DOS برای همین نسل اولیه طراحی شده بود و به خدمات BIOS تکیه داشت؛ بنابراین در مد حقیقی اجرا می‌شد.

نقص اصلی مد حقیقی این است که فقط می‌تواند به ۱ مگابایت حافظه دسترسی داشته باشد و هیچ مکانیزمی برای حفاظت حافظه یا چندوظیفگی ندارد. در نتیجه، اگر برنامه‌ای اشتباه کند، می‌تواند حافظه یا داده‌های دیگر برنامه‌ها را تخریب کند.

در پردازنده‌های دیگر مانند ARM یا RISC-V چنین حالتی وجود ندارد و آن‌ها مستقیماً پس از روشن شدن وارد حالتی می‌شوند که امکان مدیریت حافظه، سطوح دسترسی و حفاظت را دارد. این معماری‌ها از ابتدا برای سیستم عامل‌های مدرن طراحی شده‌اند، بنابراین نیازی به حالت‌های سازگاری قدیمی مانند مد حقیقی ندارند.

18- یکی دیگر در از مدهای مهم، مد حفاظت شده میباشد. وظیفه ی اصلی این مود چیست؟ پردازنده ها در چه زمانی در این مود قرار میگیرند؟

مد حفاظت شده در پردازنده های 86x برای فراهم کردن قابلیت های پیشرفته مانند مدیریت حافظه، چندوظیفگی و حفاظت بین پردازنده ها به کار می رود. در این مد، پردازنده می تواند از حافظه مجازی استفاده کند و برای هر پردازنده فضای حافظه جداگانه ای در نظر بگیرد تا برنامه ها نتوانند به حافظه یکدیگر دسترسی پیدا کنند. پردازنده پس از انجام مراحل اولیه بوت و راه اندازی توسط بوت لودر، زمانی که سیستم عامل شروع به اجرا می کند، از مد حقیقی به مد حفاظت شده منتقل می شود. در این حالت، کنترل کامل سخت افزار به سیستم عامل سپرده می شود و سیستم آماده اجرای برنامه های کاربر و مدیریت منابع به صورت ایمن می گردد.

پس از شروع در مد حقیقی، بوت لودر یا هسته ی اولیه با آماده سازی جدول توصیفگر (GDT)، بارگذاری آن با دستور lgdt و سپس تنظیم بیت PE در ثبات 0CR پردازنده را به مد حفاظت شده می برد؛ در ادامه با تنظیم 4CR3/CR و بیت PG در 0CR می توان صفحه بندی را نیز فعال کرد.

19- کد bootmain.c، هسته را با شروع از یک سکتور پس از سکتور بوت، خوانده و در آدرس 100000x0 قرار میدهد. علت انتخاب این آدرس چیست؟ چرا این آدرس از 0 شروع نشده است؟

آدرس 100000x0 برابر با ۱ مگابایت است و به عنوان نقطه شروع بارگذاری هسته در بسیاری از سیستم ها انتخاب می شود، چون این ناحیه از حافظه معمولاً آزاد و مخصوص سیستم عامل است. در معماری 86x، فضای زیر ۱ مگابایت برای بایوس، وقفه ها، داده های سیستم و دستگاه های ورودی خروجی رزرو شده است. اگر هسته از آدرس صفر شروع می شد، با این بخش ها تداخل پیدا می کرد و سیستم نمی توانست به درستی بوت شود. بنابراین، آدرس 100000x0 انتخاب شده تا از محدوده رزرو شده بایوس و داده های سخت افزار جدا باشد و فضای کافی برای بارگذاری کامل هسته سیستم عامل فراهم شود.

- اشکال زدایی:

با اجرای این سه دستور gdb روی سیستم عامل اجرا می شود:

1- در یک ترمینال دستور `gdb-qemu make` را اجرا کنید.

2- سپس در ترمینالی دیگر، فایل کد اجرایی را به عنوان ورودی به GDB بدهید

3- با وارد کردن دستور `tcp remote target26000::` در GDB، اتصال به سیستم عامل صورت می گیرد.

```

*** Now run 'gdb'.
qemu-system-i386 -serial mon:stdio -drive file=fs.img,index=1,media=disk,format=raw -drive file=xv6.img,index=0,media=disk,format=raw -smp 2 -m 512 -S -gdb tcp::26000
xv6...
cpu0: starting 0
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap start 58
init: starting sh

```

آدرس 0x0000ffff همان نقطه‌ی شروع (Reset Vector) بایوس است که با دادن c به عنوان ورودی به کنسول gdb بوت سیستم عامل کامل می‌شود:

```

(gdb) info (gdb)/Auto loading safe path
(gdb) file kernel
Reading symbols from kernel...done.
(gdb) target remote tcp::26000
Remote debugging using tcp::26000
0x0000ffff in ?? ()
(gdb) c
Continuing.

```

- روند اجرای GDB:

20- برای مشاهده Breakpoint ها از چه دستوری استفاده میشود؟

Info breakpoints- این دستور فهرستی از تمام breakpoint های فعال، شماره آن‌ها، محل قرارگیری درکد و وضعیت فعال یا غیرفعال بودنشان را نمایش می‌دهد.

```

(gdb) info break
Num      Type           Disp Enb Address      What
1        breakpoint     keep y   0x80103030 in main at main.c:20
2        breakpoint     keep y   0x00007c00
(gdb) |

```

21- برای حذف یک Breakpoint از چه دستوری و چگونه استفاده میشود؟

delete [number Breakpoint]

اگر شماره‌ای مشخص نکنیم تمامشان حذف می‌شوند.

```

(gdb) info break
Num      Type           Disp Enb Address      What
1        breakpoint     keep y   0x80103030 in main at main.c:20
2        breakpoint     keep y   0x00007c00
(gdb) delete 2
(gdb) delete 1
(gdb) info break
No breakpoints or watchpoints.

```

-کنترل روند اجرا و دسترسی به حالت سیستم:

22-دستور زیر را اجرا کنید. خروجی آن چه چیزی را نشان میدهد؟

```
(gdb) b main
Breakpoint 1 at 0x80103030: file main.c, line 20.
(gdb) c
Continuing.

Thread 1 hit Breakpoint 1, main () at main.c:20
20      kinit1(end, P2V(4*1024*1024)); // phys page allocator
(gdb) bt
#0  main () at main.c:20
(gdb) |
```

bt مخفف (backtrace) پشته فراخوانی فعلی را نشان می‌دهد: هر فریم شامل شماره فریم، نام تابع، فایل و شماره خط (اگر نمادها موجود باشند) است. فریم شماره 0# همان جایی است که CPU الان متوقف شده؛ فریم‌های بعدی، توابع فراخواننده‌اند. این خروجی برای فهمیدن مسیر اجرای کد تا نقطه توقف به‌کار می‌آید.

23-دو تفاوت دستورهای x و print را توضیح دهید. چگونه میتوان محتوای یک ثبات خاص را چاپ کرد؟

1. دستور print برای نمایش مقدار متغیرها یا عبارات در سطح زبان C استفاده می‌شود، در حالی که دستور x برای مشاهده حافظه خام در یک آدرس مشخص به کار می‌رود.
 2. خروجی print به صورت عددی یا متنی مطابق نوع داده در برنامه نمایش داده می‌شود، اما خروجی x معمولاً به صورت باینری یا هگزادسیمال از حافظه خام نشان داده می‌شود.
- برای چاپ مقدار یک ثبات خاص از دستور print \$register استفاده می‌شود.

```
(gdb) help x
Examine memory: x/FMT ADDRESS.
ADDRESS is an expression for the memory address to examine.
FMT is a repeat count followed by a format letter and a size letter.
Format letters are o(octal), x(hex), d(decimal), u(unsigned decimal),
t(binary), f(float), a(address), i(instruction), c(char), s(string)
and z(hex, zero padded on the left).
Size letters are b(byte), h(halfword), w(word), g(giant, 8 bytes).
The specified number of objects of the specified size are printed
according to the format. If a negative number is specified, memory is
examined backward from the address.

Defaults for format and size letters are those previously used.
Default count is 1. Default address is following last thing printed
with this command or "print".
```

```
(gdb) help print
Print value of expression EXP.
Variables accessible are those of the lexical environment of the selected
stack frame, plus all those whose scope is global or an entire file.

$NUM gets previous value number NUM. $ and $$ are the last two values.
$$NUM refers to NUM'th value back from the last one.
Names starting with $ refer to registers (with the values they would have
if the program were to return to the stack frame now selected, restoring
all registers saved by frames farther in) or else to debugger
"convenience" variables (any such name not a known register).
Use assignment expressions to give values to convenience variables.

{TYPE}ADREXP refers to a datum of data type TYPE, located at address ADREXP.
@ is a binary operator for treating consecutive data objects
anywhere in memory as an array. FOO@NUM gives an array whose first
element is FOO, whose second element is stored in the space following
where FOO is stored, etc. FOO must be an expression whose value
resides in memory.

EXP may be preceded with /FMT, where FMT is a format letter
but no count or size letter (see "x" command).
```

24- برای نمایش وضعیت ثبات ها از چه دستوری استفاده می شود؟ برای متغیرهای محلی چطور؟ نتیجه این دستور را در گزارش کار خود بیاورید. همچنین در گزارش خود توضیح دهید که در معماری 86x رجیستر های edi و esi نشانگر چه چیزی هستند؟

برای نمایش وضعیت همه ثبات ها از دستور info registers استفاده می شود.

در معماری 86x، ثبات edi معمولاً به عنوان مقصد عملیات در توابعی مانند کپی یا جابه جایی داده استفاده می شود، در حالی که ثبات esi نقش مبدأ داده را دارد. esi آدرس داده ای است که باید خوانده شود و edi آدرسی است که داده در آن نوشته می شود.

```
(gdb) info registers
eax            0x80103030      -2146422736
ecx            0x0            0
edx            0x1f0          496
ebx            0x10094         65684
esp            0x801154d0      0x801154d0
ebp            0x7bf8          0x7bf8
esi            0x10094         65684
edi            0x0            0
eip            0x80103030      0x80103030 <main>
eflags         0x86           [ IOPL=0 SF PF ]
cs             0x8            8
ss             0x10           16
ds             0x10           16
es             0x10           16
fs             0x0            0
gs             0x0            0
fs_base        0x0            0
gs_base        0x0            0
k_gs_base      0x0            0
cr0            0x80010011      [ PG WP ET PE ]
cr2            0x0            0
cr3            0x109000        [ PDBR=0 PCID=0 ]
cr4            0x10           [ PSE ]
cr8            0x0            0
efer           0x0            [ ]
xmm0           {v4_float = {0x0, 0x0, 0x0, 0x0}, v2_double = {0x0, 0x0}, v16_int8 = {0x0 <repeats 16 times>},
v8_int16 = {0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}, v4_int32 = {0x0, 0x0, 0x0, 0x0}, v2_int64 = {0x0, 0x0},
uint128 = 0x0}
```

برای نمایش مقدار متغیرهای محلی از دستور `info locals` استفاده می شود.

25- به کمک استفاده از GDB، درباره ساختار `struct input` موارد زیر را توضیح دهید:

- توضیح کلی این `struct` و متغیرهای درونی آن و نقش آنها.
- نحوه و زمان تغییر مقدار متغیرهای درونی.

```
(gdb) break consoleintr
Note: breakpoint 1 also set at pc 0x801008a0.
Breakpoint 2 at 0x801008a0: file console.c, line 196.
(gdb) commands
Type commands for breakpoint(s) 2, one per line.
End with a line saying just "end".
>silent
>printf "r=%d w=%d e=%d\n", input.r, input.w, input.e
>continue
>end
(gdb) Quit (expect signal SIGINT when the program is resumed)
(gdb) continue
Continuing.
```

`struct input` برای مدیریت ورودی کاربر استفاده می شود، به ویژه داده های دریافتی از کیبورد

```
Thread 1 hit Breakpoint 1, consoleintr (getc=0x80105b90 <uartgetc>) at console.c:196
196      acquire(&cons.lock);
r=0 w=24 e=24
```

پس از وارد کردن یک دستور:

```
Thread 1 hit Breakpoint 1, consoleintr (getc=0x80105b90 <uartgetc>) at console.c:196
196      acquire(&cons.lock);
r=35 w=38 e=38
```

—[INPUT_BUF] Buf بافر خط ورودی

این آرایه تمام کاراکترهایی را که از صفحه‌کلید می‌آیند نگه می‌دارد. درج کاراکترها در اندیسی انجام می‌شود که از `e` به‌دست می‌آید: `[buf[e % INPUT_BUF]]`. وقتی کاراکتر معمولی تایپ شود، مقدار ASCII همان کاراکتر در این آرایه نوشته می‌شود؛ با Backspace آخرین عنصر مؤثر پاک تلقی می‌شود (نمایش روی صفحه هم اصلاح می‌گردد). هنگام خواندن از کنسول با سیستم‌کال `read`، کاراکترها از موقعیت `r` برداشته می‌شوند

```
#define INPUT_BUF 128
struct {
    char buf[INPUT_BUF];
    uint r; // Read index
    uint w; // Write index
    uint e; // Edit index
} input;
```

—`r` اندیس خواندن (read index):

`r` جایی را نشان می‌دهد که مصرف‌کننده (فرآیند خواننده ترمینال) از آن کاراکتر بعدی را دریافت می‌کند. هر بار که `read` کاراکتری تحویل می‌گیرد، `r` یک واحد افزایش می‌یابد (`++r`). وقتی `r == w` شود یعنی دیگر داده «آماده تحویل» نداریم و `read` باید منتظر خط بعدی بماند. به‌دلیل حلقوی بودن بافر، کاربرد واقعی اندیس همیشه `r % INPUT_BUF` است. مقدار `r` فقط در مسیر مصرف تغییر می‌کند، نه هنگام تایپ.

— `w` اندیس «آماده برای خواندن» (write index):

`w` انتهای بخشی از بافر را نشان می‌دهد که برای فرآیندهای کاربر آماده خواندن است. این مقدار وقتی تغییر می‌کند که یک «خط» کامل شود؛ یعنی کاربر `Enter` بزند (`'n'`) یا علامت پایان ورودی (`Ctrl-D`) ارسال کند. در این لحظه، `w = e` را می‌گذارد و با `wakeup(&input.r)` خواننده‌ها را از خواب بیدار می‌کند تا داده را تحویل بگیرند. در حین تایپ عادی (قبل از پایان خط)، `w` ثابت می‌ماند تا از دیدشدن نیمی از خط توسط خواننده جلوگیری شود؛ بعد از مصرف کامل داده‌ها، با افزایش `r` به `w` می‌رسد.

— `e` اندیس ویرایش (edit index)

e انتهای محتوای در حال ویرایش کاربر است. با هر کاراکتر معمولی که تایپ می‌شود یک واحد زیاد می‌شود (e++) و مقدار آن در [buf[e % INPUT_BUF] نوشته می‌شود؛ با Backspace یا Ctrl-H یک واحد کم می‌شود (e--) تا آخرین ورودی پاک شود؛ با Ctrl-U (kill line) مقدار e مستقیماً به w برمی‌گردد تا کل بخش ویرایش شده فعلی حذف شود.

- اشکال زدایی در سطح کد اسمبلی:

26- خروجی دستورات src layout و asm layout در TUI چیست؟

با وارد کردن دستور src layout می‌توانیم کد C مربوطه را هنگام توقف و دیباگ در محیط TUI ببینیم.

```

17 int
18 main(void)
19 {
B+> 20 kinit1(end, P2V(4*1024*1024)); // phys page allocator
21 kvmalloc(); // kernel page table
22 mpinit(); // detect other processors
23 lapicinit(); // interrupt controller
24 seginit(); // segment descriptors
25 picinit(); // disable pic
26 ioapicinit(); // another interrupt controller
27 consoleinit(); // console hardware
28 uartinit(); // serial port
29 pinit(); // process table

```

با وارد کردن دستور asm layout می‌توانیم کد اسمبلی را هنگام توقف و دیباگ در محیط TUI ببینیم.

```

B+> 0x80103030 <main> lea ecx,[esp+0x4]
0x80103034 <main+4> and esp,0xfffffffff0
0x80103037 <main+7> push DWORD PTR [ecx-0x4]
0x8010303a <main+10> push ebp
0x8010303b <main+11> mov ebp,esp
0x8010303d <main+13> push ebx
0x8010303e <main+14> push ecx
0x8010303f <main+15> sub esp,0x8
0x80103042 <main+18> push 0x80400000
0x80103047 <main+23> push 0x801154d0
0x8010304c <main+28> call 0x801025f0 <kinit1>
0x80103051 <main+33> call 0x80106ea0 <kvmalloc>
0x80103056 <main+38> call 0x801031e0 <mpinit>
0x8010305b <main+43> call 0x801027e0 <lapicinit>

```

27- برای جابه جایی میان توابع زنجیره فراخوانی جاری از چه دستوراتی استفاده می‌شود؟

<frame <n>: رفتن مستقیم به فریم شماره (انتخاب یک فریم مشخص).

[up [n]: حرکت به سمت فریم‌های بالاتر (caller)؛ با گزینهٔ n چند فریم یکجا بالا می‌رود.

[down [n]: حرکت به سمت فریم‌های پایین‌تر، با گزینهٔ n چند فریم یکجا پایین می‌رود.